

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_191a9b86-991\agent-a7110ae18cf1aa1f8.jsonl`
- SHA-256: `b49aeb35b0c6bce317f656cbf0f6bf97215244f7cc814edf47b0caa8ee18449c`
- Source modified: `2026-06-22T03:49:45+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_191a9b86-991`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T03:49:01.182Z)

Adversarially verify this P2c review finding by reading the cited code in C:\\Users\\avidu\\Projects\\khelsutra-p2c yourself (run `cd C:\\Users\\avidu\\Projects\\khelsutra-p2c && git diff origin/master...HEAD`). Is it a REAL defect to fix, or a false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: i18n-integration
Severity(claimed): nit
Title: New SetupWizard.tsx is outside the no-bare-strings i18n guard's FILES list
File: web/src/i18n/hygiene.test.ts:105-115
Detail: The no-bare-strings guard (asserts placeholder/aria-label/title attributes route through t()) enumerates a fixed FILES list that was not extended to include components/SetupWizard.tsx. SetupWizard.tsx currently does the right thing (aria-label={t("wizard.title")}) at SetupWizard.tsx:122, no hardcoded user-facing attribute), so there is no live regression ? but a future hardcoded aria-label/title/placeholder added to the wizard would not be caught. Minor coverage gap, in the same spirit as the over-claimed touch coverage above.
Fix: Add "components/SetupWizard.tsx" to the FILES array in the no-bare-strings describe block so the new component is also guarded.

2. assistant (2026-06-22T03:49:03.590Z)

I'll verify this finding by examining the cited code in the khelsutra-p2c worktree.

3. user (2026-06-22T03:49:04.940Z)

```
locales/da/common.json          | 32 +++++
locales/en/common.json          | 32 +++++
locales/es/common.json          | 32 +++++
locales/hi/common.json          | 32 +++++
locales/id/common.json          | 32 +++++
locales/kn/common.json          | 32 +++++
site/public/locales/da/common.json | 32 +++++
site/public/locales/en/common.json | 32 +++++
site/public/locales/es/common.json | 32 +++++
site/public/locales/hi/common.json | 32 +++++
site/public/locales/id/common.json | 32 +++++
site/public/locales/kn/common.json | 32 +++++
web/src/App.tsx                  | 9 +-
web/src/api/client.ts            | 6 ++
web/src/api/types.ts             | 17 +++
web/src/components/SetupWizard.test.tsx | 115 +++++
web/src/components/SetupWizard.tsx | 179 +++++
web/src/components/a4.touch.test.tsx | 4 +
18 files changed, 713 insertions(+), 1 deletion(-)
```

4. assistant (2026-06-22T03:49:07.287Z)

The finding cites specific lines. Let me read the hygiene test and the wizard component.

5. user (2026-06-22T03:49:08.246Z)

```
1      // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2      // 1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3      // drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4      // starts enforcing the moment a kn/hi catalog lands (B5/B6).
5      // 2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6      // aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7      // the doc; an AST-level sweep of JSX text can extend it later.
8      import { describe, it, expect } from "vitest";
9      import { readdirSync, readFileSync } from "node:fs";
10     import { join } from "node:path";
11
12     // Catalogs are hoisted to repo-root /locales (D-CATALOG-HOME): web/src/i18n ->
../.../locales.
13     const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
14     const SRC_DIR = join(import.meta.dirname, "..");
15
16     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
17         const keys: string[] = [];
18         for (const [k, v] of Object.entries(obj)) {
19             const key = prefix ? `${prefix}.${k}` : k;
20             if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
21             else keys.push(key);
22         }
23         return keys.sort();
24     }
25
26     const loadKeys = (locale: string): string[] =>
27         flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
28
29     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
30         const enKeys = loadKeys("en");
31         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
32             .filter((d) => d.isDirectory() && d.name !== "en")
33             .map((d) => d.name);
34
35         it("en is the non-empty source catalog", () => {
36             expect(enKeys.length).toBeGreaterThan(20);
37         });
38
39         for (const loc of others) {
40             it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
41                 const keys = loadKeys(loc);
42                 const missing = enKeys.filter((k) => !keys.includes(k));
43                 const extra = keys.filter((k) => !enKeys.includes(k));
44                 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
45             });
46         }
47     });
48
49     // Key-parity proves every locale has the same KEYS; this proves every locale's VALUE
keeps the same
50     // simple `{placeholder}` set as en ? so a translation can never silently drop a
```

```

{count}/{error}/{min}/
51 // {filename}/{n}/? and break interpolation. (ICU plural headers like `{count, plural,
?}` carry no
52 // simple {word} placeholder and match as empty on both sides, so they're correctly
ignored.)
53 describe("placeholder parity ? every locale preserves en's {placeholders}", () => {
54   function flatEntries(obj: Record<string, unknown>, prefix = ""): Record<string,
string> {
55     const out: Record<string, string> = {};
56     for (const [k, v] of Object.entries(obj)) {
57       const key = prefix ? `${prefix}.${k}` : k;
58       if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
59       else out[key] = String(v);
60     }
61     return out;
62   }
63   const placeholders = (s: string) => (s.match(/\{(\w+)\}/g) ?? []).sort();
64   const load = (loc: string) => flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR,
loc, "common.json"), "utf8")));
65   const enE = load("en");
66   const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
67     .filter((d) => d.isDirectory() && d.name !== "en")
68     .map((d) => d.name);
69
70   for (const loc of others) {
71     it(`${loc} preserves every {placeholder} that en uses`, () => {
72       const locE = load(loc);
73       const mismatches: Array<{ key: string; en: string[]; got: string[] }> = [];
74       for (const key of Object.keys(enE)) {
75         if (!(key in locE)) continue; // missing keys are the key-parity test's job
76         const a = placeholders(enE[key]);
77         const b = placeholders(locE[key]);
78         if (a.join(",") !== b.join(",")) mismatches.push({ key, en: a, got: b });
79       }
80       expect(mismatches).toEqual([]);
81     });
82   }
83 });
84
85 describe("es 3-form plural (CLDR one/many/other) ? MULTILINGUAL Phase A", () => {
86   // Spanish cardinal is {one, many, other}; every es ICU plural message MUST carry a
`many` arm
87   // (the `many` category fires on large/compact numbers) or intl-messageformat silently
falls back.
88   // These are the plural-bearing keys in the catalog.
89   const es = JSON.parse(readFileSync(join(LOCALES_DIR, "es", "common.json"), "utf8")) as
Record<string, Record<string, string>>;
90   const pluralKeys: Array<[string, string]> = [
91     ["footer", "count"],
92     ["footer", "floorSomeLead"],
93     ["selection", "count"],
94   ];
95   for (const [ns, key] of pluralKeys) {
96     it(`es ${ns}.${key} carries one/many/other arms`, () => {
97       const msg = es[ns][key];
98       expect(msg, `${ns}.${key}`).toMatch(/bone\s*\{/);
99       expect(msg, `${ns}.${key}`).toMatch(/bmany\s*\{/);
100       expect(msg, `${ns}.${key}`).toMatch(/bother\s*\{/);
101     });
102   }
103 });
104
105 describe("no-bare-strings ? user-facing component attributes go through t()", () => {
106   const FILES = [

```

```

107     "App.tsx",
108     "components/QueryBar.tsx",
109     "components/TapFilters.tsx",
110     "components/RallyGrid.tsx",
111     "components/SelectionFooter.tsx",
112     "components/CompileResult.tsx",
113     "components/LanguageSwitcher.tsx",
114     "components/IngestPanel.tsx",
115 ];
116 // A hardcoded placeholder / aria-label / title with letters is a bare-string
regression ? it
117 // must be an expression ({t(...)}), not a quoted literal.
118 const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"^[A-Za-z]{2}[^"]*"$/g;
119
120 for (const f of FILES) {
121     it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
122         const src = readFileSync(join(SRC_DIR, f), "utf8");
123         expect(src.match(BARE_ATTR) ?? []).toEqual([]);
124     });
125 }
126 });
127
128 describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
129     it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google
Fonts)", () => {
130         const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
131         expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
132         expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
133     });
134     it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
135         const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
136         expect(css).toMatch(/Noto Sans Kannada Variable/);
137         expect(css).toMatch(/Noto Sans Devanagari Variable/);
138     });
139 });
140
141 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
internet) ?
142 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
CDN/Google Fonts.
143 describe("offline / self-contained ? no external runtime assets", () => {
144     const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr\bcdn\.[a-z]/i;
145     const FILES = [
146         "../index.html", // web/index.html
147         "main.tsx",
148         "index.css",
149     ];
150     for (const rel of FILES) {
151         it(`${rel}: no external font/CDN reference`, () => {
152             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
153         });
154     }
155     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
156         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
157         expect(html).not.toMatch(/(?:href|src)\s*=\s*"["']https?:\/\//i);
158     });
159 });
160

```

6. user (2026-06-22T03:49:08.724Z)

```

1 // First-run setup wizard (PACKAGING_PLAN.md P2c). Three quick steps ? AI key / compute
/ videos ?
2 // rendered from the engine's GET /api/capabilities (an honest probe of what THIS box

```

```

can do).
3      // Localized (en/kn/hi/es/da/id). The Gemini-key step is GUIDED setup, not auto-save:
the engine has
4      // no key-write endpoint yet, so we tell the user how to add the key (offline `motion`
works with none).
5      import { useEffect, useState } from "react";
6      import { useTranslation } from "react-i18next";
7      import { getCapabilities } from "../api/client";
8      import type { EngineCapabilities } from "../api/types";
9      import { log, newCorrelationId } from "../lib/log";
10
11     /** localStorage flag ? distinct from the language key ? so the wizard shows ONCE per
browser. */
12     export const WIZARD_DONE_KEY = "khelsutra.wizard.completed";
13
14     export function isWizardDone(): boolean {
15         try {
16             return localStorage.getItem(WIZARD_DONE_KEY) === "true";
17         } catch {
18             return false; // private-mode / no storage ? show it (harmless; user can skip)
19         }
20     }
21
22     const STEPS = 3;
23
24     function KeyStep({ caps }: { caps: EngineCapabilities | null }) {
25         const { t } = useTranslation();
26         const present = caps?.gemini_key_present === true;
27         return (
28             <div>
29                 <h3 className="font-semibold">{t("wizard.key.title")}</h3>
30                 {present ? (
31                     <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
32                 ) : (
33                     <>
34                         <p className="mt-2 text-ink/80">{t("wizard.key.missing")}</p>
35                         <p className="mt-2 text-sm text-ink/70">{t("wizard.key.how")}</p>
36                         <a
37                             href="https://aistudio.google.com/apikey"
38                             target="_blank"
39                             rel="noreferrer"
40                             className="mt-2 inline-block text-sm font-medium text-green underline"
41                         >
42                             {t("wizard.key.getKey")}
43                         </a>
44                     </>
45                 )}
46             </div>
47         );
48     }
49
50     function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
51         const { t } = useTranslation();
52         const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
53         const gpu = caps?.gpu?.nvidia_smi_present === true;
54         const detector = caps?.default_segmenter ?? "motion";
55         return (
56             <div>
57                 <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
58                 <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
59                 <ul className="mt-3 space-y-1 text-sm text-ink/70">
60                     <li>{t("wizard.compute.detector", { name: detector })}</li>
61                     <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
t("wizard.compute.ffmpegMissing")}</li>
62                     <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>

```

```

63         </ul>
64     </div>
65 );
66 }
67
68 function VideosStep() {
69     const { t } = useTranslation();
70     return (
71         <div>
72             <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
73             <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
74             <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
75         </div>
76     );
77 }
78
79 export function SetupWizard({ onClose }: { onClose: () => void }) {
80     const { t } = useTranslation();
81     const [step, setStep] = useState(0);
82     const [caps, setCaps] = useState<EngineCapabilities | null>(null);
83     const [loading, setLoading] = useState(true);
84     const [offline, setOffline] = useState(false);
85
86     useEffect(() => {
87         let alive = true;
88         const cid = newCorrelationId();
89         getCapabilities(cid)
90             .then((c) => {
91                 if (!alive) return;
92                 setCaps(c);
93                 setLoading(false);
94             })
95             .catch((err) => {
96                 if (!alive) return;
97                 // Engine not up yet ? don't assert false capabilities; offer a graceful skip.
98                 setOffline(true);
99                 setLoading(false);
100                 log("warn", "wizard.capabilities_unavailable", { cid, error: String(err) });
101             });
102         return () => {
103             alive = false;
104         };
105     }, []);
106
107     const finish = () => {
108         try {
109             localStorage.setItem(WIZARD_DONE_KEY, "true");
110         } catch {
111             /* private mode: still close (the wizard just reappears next visit) */
112         }
113         onClose();
114     };
115
116     const stepping = !loading && !offline; // the 3-step flow only when capabilities
loaded
117     const isLast = step === STEPS - 1;
118
119     return (
120         <section
121             role="dialog"
122             aria-label={t("wizard.title")}
123             className="mt-8 rounded-2xl border border-ink/10 bg-white p-6 shadow-sm"
124         >
125             <div className="flex items-center justify-between gap-3">
126                 <h2 className="text-xl font-bold">{t("wizard.title")}</h2>

```

```

127         <button
128             type="button"
129             onClick={finish}
130             className="min-h-11 rounded-lg px-3 text-sm text-ink/60 hover:text-ink"
131         >
132             {t("wizard.skip")}
133         </button>
134     </div>
135     <p className="mt-1 text-sm text-ink/70">{t("wizard.intro")}</p>
136     {stepping && (
137         <p className="mt-1 text-xs font-medium text-ink/50">{t("wizard.step", { n: step
+ 1 })}</p>
138     )}
139
140     <div className="mt-5 min-h-[128px]" aria-live="polite">
141         {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142         {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143         {stepping && step === 0 && <KeyStep caps={caps} />}
144         {stepping && step === 1 && <ComputeStep caps={caps} />}
145         {stepping && step === 2 && <VideosStep />}
146     </div>
147
148     <div className="mt-6 flex items-center justify-between">
149         {stepping ? (
150             <>
151                 <button
152                     type="button"
153                     disabled={step === 0}
154                     onClick={() => setStep((s) => Math.max(0, s - 1))}
155                     className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"
156                 >
157                     {t("wizard.back")}
158                 </button>
159                 <button
160                     type="button"
161                     onClick={() => (isLast ? finish() : setStep((s) => Math.min(STEPS - 1, s +
1))))}
162                     className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
163                 >
164                     {isLast ? t("wizard.done") : t("wizard.next")}
165                 </button>
166             </>
167         ) : (
168             <button
169                 type="button"
170                 onClick={finish}
171                 className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
172             >
173                 {t("wizard.done")}
174             </button>
175         )}
176     </div>
177 </section>
178 );
179 }
180

```

7. assistant (2026-06-22T03:49:14.889Z)

Now let me verify the finding's claims:

1. The FILES list at lines 106-115 does NOT include `components/SetupWizard.tsx` ? confirmed.

2. SetupWizard.tsx uses `aria-label={t("wizard.title")}` at line 122 ? confirmed (it's an expression, not a bare literal).
3. No hardcoded user-facing attributes in SetupWizard.tsx ? let me verify there are no bare placeholder/aria-label/title literals anywhere in the file.

Looking at SetupWizard.tsx: the only attributes matching the guard's regex (`placeholder|aria-label|title`) are `aria-label={t("wizard.title")}` (line 122, expression, OK) and `aria-live="polite"` (line 140 ? not matched by the regex; `aria-live` ? `aria-label`). The `title` keyword: there's `wizard.title` keys but no JSX `title=` attribute. So no live regression. Let me double-check there's no `title=` or `placeholder=` JSX attribute.

8. user (2026-06-22T03:49:16.243Z)

No matches found

9. assistant (2026-06-22T03:49:20.796Z)

Confirmed: no bare-literal placeholder/aria-label/title attributes in SetupWizard.tsx, so there is no live regression. The finding's "no live regression" characterization holds.

Now let me check whether the other new file in the diff (IngestPanel.tsx is already in the list; let me check if there's a pattern of how new components get added to the guard). Let me also verify the finding's claim about the "over-claimed touch coverage above" referenced in the detail, and confirm SetupWizard.test.tsx exists (the component IS tested), to understand whether the wizard is genuinely under-guarded or whether its own test file already covers i18n behavior.

10. user (2026-06-22T03:49:21.755Z)

```
1      // @vitest-environment jsdom
2      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
3      import { render, screen, fireEvent, cleanup, waitFor } from "@testing-library/react";
4      import i18n from "../i18n/index";
5      import { SetupWizard, WIZARD_DONE_KEY, isWizardDone } from "../SetupWizard";
6      import type { EngineCapabilities } from "../api/types";
7
8      function stubCapabilities(caps: EngineCapabilities | "reject") {
9          vi.stubGlobal(
10              "fetch",
11              vi.fn(async (url: string | URL) => {
12                  const u = typeof url === "string" ? url : url.toString();
13                  if (u.endsWith("/api/capabilities")) {
14                      if (caps === "reject") throw new TypeError("Failed to fetch");
15                      return { ok: true, status: 200, json: async () => caps } as unknown as Response;
16                  }
17                  return { ok: false, status: 404, text: async () => "" } as unknown as Response;
18              }),
19          );
20      }
21
22      const LIGHT: EngineCapabilities = {
23          gemini_key_present: false,
24          default_segmenter: "motion",
25          ffmpegpeg: { ffmpegpeg: false, ffprobe: false },
26          gpu: { nvidia_smi_present: false },
27      };
28      const READY: EngineCapabilities = {
29          gemini_key_present: true,
30          default_segmenter: "gemini",
31          ffmpegpeg: { ffmpegpeg: true, ffprobe: true },
32          gpu: { nvidia_smi_present: true },
33      };
34
35      beforeEach(async () => {
36          window.localStorage.clear();
```

```

37     await i18n.changeLanguage("en");
38   });
39   afterEach(() => {
40     cleanup();
41     vi.restoreAllMocks();
42     vi.unstubAllGlobals();
43   });
44
45   describe("SetupWizard", () => {
46     it("checks capabilities, then step 1 guides a missing Gemini key (offline-first
default)", async () => {
47       stubCapabilities(LIGHT);
48       render(<SetupWizard onClose={() => {}} />);
49       expect(screen.getByText(/Checking what this device can do/)).toBeTruthy();
50       expect(await screen.findByText(/built-in motion detector/)).toBeTruthy();
51       expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
52       expect(screen.getByText(/Step 1 of 3/)).toBeTruthy();
53     });
54
55     it("walks all 3 steps from real capabilities; Back returns; Done sets the flag +
closes", async () => {
56       stubCapabilities(LIGHT);
57       const onClose = vi.fn();
58       render(<SetupWizard onClose={onClose} />);
59       await screen.findByText(/built-in motion detector/);
60
61       fireEvent.click(screen.getByText("Next")); // ? compute
62       expect(screen.getByText("Detector: motion")).toBeTruthy();
63       expect(screen.getByText(/ffmpeg not found/)).toBeTruthy();
64       expect(screen.getByText(/No GPU needed/)).toBeTruthy();
65
66       fireEvent.click(screen.getByText("Back")); // ? key
67       expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
68
69       fireEvent.click(screen.getByText("Next")); // ? compute
70       fireEvent.click(screen.getByText("Next")); // ? videos
71       expect(screen.getByText(/You're all set/)).toBeTruthy();
72
73       fireEvent.click(screen.getByText("Get started")); // Done on last step
74       expect(onClose).toHaveBeenCalledTimes(1);
75       expect(window.localStorage.getItem(WIZARD_DONE_KEY)).toBe("true");
76     });
77
78     it("reflects a connected key + a GPU/ffmpeg-ready box", async () => {
79       stubCapabilities(READY);
80       render(<SetupWizard onClose={() => {}} />);
81       expect(await screen.findByText(/Gemini is connected/)).toBeTruthy();
82       fireEvent.click(screen.getByText("Next"));
83       expect(screen.getByText("Detector: gemini")).toBeTruthy();
84       expect(screen.getByText(/Video tools \ffmpeg\ ready/)).toBeTruthy();
85       expect(screen.getByText(/A GPU was detected/)).toBeTruthy();
86     });
87
88     it("Skip closes + sets the flag without finishing the steps", async () => {
89       stubCapabilities(LIGHT);
90       const onClose = vi.fn();
91       render(<SetupWizard onClose={onClose} />);
92       await screen.findByText(/built-in motion detector/);
93       fireEvent.click(screen.getByText("Skip"));
94       expect(onClose).toHaveBeenCalledTimes(1);
95       expect(isWizardDone()).toBe(true);
96     });
97
98     it("degrades gracefully when the engine is unreachable (offline) ? message + a Done
escape", async () => {

```

```

99     stubCapabilities("reject");
100     const onClose = vi.fn();
101     render(<SetupWizard onClose={onClose} />);
102     expect(await screen.findByText(/Couldn't reach KhelSutra/)).toBeTruthy();
103     // No step content / step indicator when offline.
104     expect(screen.queryByText(/Step 1 of 3/)).toBeNull();
105     fireEvent.click(screen.getByText("Get started"));
106     await waitFor(() => expect(onClose).toHaveBeenCalled());
107     expect(isWizardDone()).toBe(true);
108   });
109
110   it("isWizardDone reads the localStorage flag", () => {
111     expect(isWizardDone()).toBe(false);
112     window.localStorage.setItem(WIZARD_DONE_KEY, "true");
113     expect(isWizardDone()).toBe(true);
114   });
115 });
116

```

11. user (2026-06-22T03:49:22.054Z)

```

1    // @vitest-environment jsdom
2    //
3    // A4 ? non-tech-savvy mobile UX. Two guarantees a phone user depends on:
4    //   1. TOUCH TARGETS ? 44px (WCAG 2.5.5 / platform HIG) on every primary control ?
pinned via the
5    //     Tailwind min-h-11 (=2.75rem=44px) / h-11 class so a future "tighten the padding"
can't
6    //     silently shrink a tap target below the floor.
7    //   2. The "not available yet" REASON is VISIBLE text, not a hover-only title ? a touch
device never
8    //     surfaces `title=`, so the explanation must render in the DOM for a phone user to
read it.
9    import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
10   import { render, screen, cleanup, fireEvent } from "@testing-library/react";
11   import i18n from "../i18n/index";
12   import App from "../App";
13   import { QueryBar } from "../QueryBar";
14   import { WIZARD_DONE_KEY } from "../SetupWizard";
15
16   // 44px floor: min-h-11 (buttons/inputs) or an explicit h-11 square (the ? dismiss).
17   const TAP_44 = /\b(?:min-h-11|h-11)\b/;
18
19   beforeEach(async () => {
20     vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
21     window.localStorage.clear();
22     // Skip the first-run wizard (P2c) so these tests render the app proper (IngestPanel +
grid) whose
23     // touch targets they assert ? the wizard's own ?44px targets are covered in
SetupWizard.test.tsx.
24     window.localStorage.setItem(WIZARD_DONE_KEY, "true");
25     await i18n.changeLanguage("en");
26   });
27
28   afterEach(() => {
29     cleanup();
30     vi.unstubAllGlobals();
31   });
32
33   describe("A4: touch targets ? 44px on primary controls", () => {
34     it("the always-visible buttons carry a ?44px tap class", () => {
35       render(<App />);
36       for (const name of [
37         "Select all",
38         "Clear",

```

```

39     "Invert",
40     "Apply", // QueryBar submit
41     "Process video", // IngestPanel submit
42     "English", // LanguageSwitcher
43     "?????",
44     "??????",
45   ]) {
46     const btn = screen.getByRole("button", { name });
47     expect(btn.className, `${name} must be ?44px`).toMatch(TAP_44);
48   }
49   // The Build button label includes the live selection count ("Build reel (6)").
50   expect(screen.getByRole("button", { name: /Build reel/
51 }).className).toMatch(TAP_44);
52 });
53 it("the text inputs carry a ?44px tap class", () => {
54   render(<App />);
55   expect(screen.getByRole("textbox", { name: "Filter rallies by length, count or
56 order" })).className).toMatch(TAP_44);
57   expect(screen.getByRole("textbox", { name: "Reel name"
58 }).className).toMatch(TAP_44);
59   expect(
60     screen.getByRole("textbox", { name: "Video location ? a gs:// bucket URL or a path
61 on the engine host" })).className,
62   ).toMatch(TAP_44);
63 });
64 it("the sample-query chips are tappable (?44px)", () => {
65   render(<App />);
66   expect(screen.getByRole("button", { name: "over 10s" })).className).toMatch(TAP_44);
67   expect(screen.getByRole("button", { name: "top 15 longest"
68 }).className).toMatch(TAP_44);
69 });
70 describe("A4: per-card shot-count confidence is visible, not hover-only", () => {
71   it("renders the confidence inline on the chip (phones never see a title=)", () => {
72     const { container } = render(<App />); // demo rallies include a High-confidence
73     expect(container.textContent).toContain("confidence: High");
74   });
75 });
76 describe("A4: the 'not available yet' reason is visible, not hover-only", () => {
77   const HELP = "Shot type, players and score aren't available yet ? filter by rally
78 length instead.";
79   it("hidden until an unsupported concept is typed, then shown as plain text", () => {
80     render(<QueryBar onApply={() => {}} />);
81     expect(screen.queryByText(HELP)).toBeNull(); // nothing typed ? no help line
82     fireEvent.change(screen.getByRole("textbox", { name: "Filter rallies by length,
83 count or order" })), {
84       target: { value: "smashes" }, // a shot-type token ? flagged "not available yet"
85     });
86     expect(screen.getByText(HELP)).toBeTruthy(); // the reason is now a visible DOM node
87   });
88 });

```

12. assistant (2026-06-22T03:49:32.318Z)

I now have a complete picture. Let me assess the finding.

The finding is factually accurate on all its claims:

1. ****FILES list does not include SetupWizard.tsx**** ? confirmed at lines 106-115. The list is ``App.tsx`, `QueryBar.tsx`, `TapFilters.tsx`, `RallyGrid.tsx`, `SelectionFooter.tsx`, `CompileResult.tsx`, `LanguageSwitcher.tsx`, `IngestPanel.tsx``. The new ``SetupWizard.tsx`` is not added.
2. ****No live regression**** ? confirmed. `SetupWizard.tsx`'s only matching attribute is ``aria-label={t("wizard.title")}`` (expression form, line 122), and my grep for bare-literal ``placeholder=|aria-label=|title=`` found zero matches. The wizard is currently fully i18n-clean.
3. ****The fix is correct**** ? adding ``"components/SetupWizard.tsx"`` to the FILES array would guard the new component.

Now the severity / is_real judgment. The guard is explicitly a "curated start" by design (the test's own header comment lines 5-7: "A curated start per the doc; an AST-level sweep of JSX text can extend it later"). The FILES list is a hand-maintained allowlist that by design must be extended as new components land. The new component is a user-facing dialog with localizable attributes (aria-label) ? exactly the kind of file the guard is meant to cover. So there IS a genuine, if minor, coverage gap: a future hardcoded aria-label/title/placeholder added to the wizard would slip through CI.

This is a real, low-stakes coverage-gap observation ? there is no current bug, and the finding itself honestly states "no live regression." It's correctly characterized as a nit. The fix is trivial, correct, and improves the guard's coverage going forward. This is not a false alarm: the claim holds up against the code. It is a true nit.

13. user (2026-06-22T03:49:40.354Z)

Structured output provided successfully

14. assistant (2026-06-22T03:49:45.084Z)

Verified. The finding is a REAL nit (not a false alarm): the new ``web/src/components/SetupWizard.tsx`` was not added to the no-bare-strings guard's ``FILES`` array in ``C:\Users\avidu\Projects\khelsutra-p2c\web\src\i18n\hygiene.test.ts`` (lines 106-115). There is no live regression ? ``SetupWizard.tsx`` correctly routes its only matching attribute through ``t()`` (``aria-label={t("wizard.title")}`` at line 122) and a targeted grep found no bare-literal placeholder/aria-label/title attributes ? but the guard is a hand-maintained allowlist that by design must be extended as new user-facing components land, so a future hardcoded localizable attribute on the wizard would slip past CI. The one-line fix (add ``"components/SetupWizard.tsx"`` to the array) is correct. Severity: nit.